

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	14
O QUE VOCÊ VIU NESTA AULA?	16
REFERÊNCIAS.....	17

EMSE

O QUE VEM POR AÍ?

Nesta aula, você aprenderá os fundamentos do Docker, um software revolucionário que tem transformado a maneira como desenvolvedores e desenvolvedoras trabalham. Além disso, vamos explorar o conceito de containers, entender suas vantagens em relação às máquinas virtuais, aprender como o Docker funciona internamente, executar comandos essenciais do Docker e aprender o conceito de volumes.

Prepare-se para uma jornada que irá mudar sua perspectiva sobre desenvolvimento e implantação de aplicações.

HANDS ON

Nesta seção, você terá a oportunidade de colocar em prática o que aprendeu sobre Docker.

Vamos executar nossos primeiros comandos Docker, criando e gerenciando containers. Além disso, você aprenderá a executar containers, listar containers ativos e inativos, entrar no modo interativo em containers já em processamento, remover containers, criar containers com volumes e executar comandos úteis para a limpeza do seu ambiente Docker.

A seguir, temos links necessários para realizar a Instalação do Docker em sua máquina (lembrando que também temos os vídeos passo a passo para instalação). Você pode seguir as instruções no site oficial do Docker.

Para instalar o Docker no Windows, clique [aqui](#). Para instalar o Docker no MacOS, clique [aqui](#). Para instalar o Docker no Ubuntu, clique [aqui](#).

SAIBA MAIS

Reforçando o conceito de containers e máquinas virtuais

Containers são uma tecnologia que permite encapsular uma aplicação e todas as suas dependências em um ambiente isolado e portátil. Isso garante que ela funcione de forma consistente em qualquer ambiente, seja no seu computador local ou em um servidor de produção.

Já Docker é uma plataforma de código aberto que automatiza o uso de containers, facilitando o desenvolvimento, o teste e a implantação de aplicações. Antes do surgimento do Docker utilizava-se máquinas virtuais, como por exemplo VirtualBox e VMware, entre outros, para executar aplicações de diversos tipos. Isso gerava um alto custo de manutenção e desenvolvimento, pois tínhamos problemas de incompatibilidade entre sistemas, versões e realização de deploy em ambiente produtivo.

Para a utilização de máquinas virtuais, necessitamos de um bom hardware visando a escalabilidade dos sistemas. Para a criação de N aplicações, precisamos de N sistemas operacionais com ambientes isolados para N aplicações, conforme a necessidade do negócio.

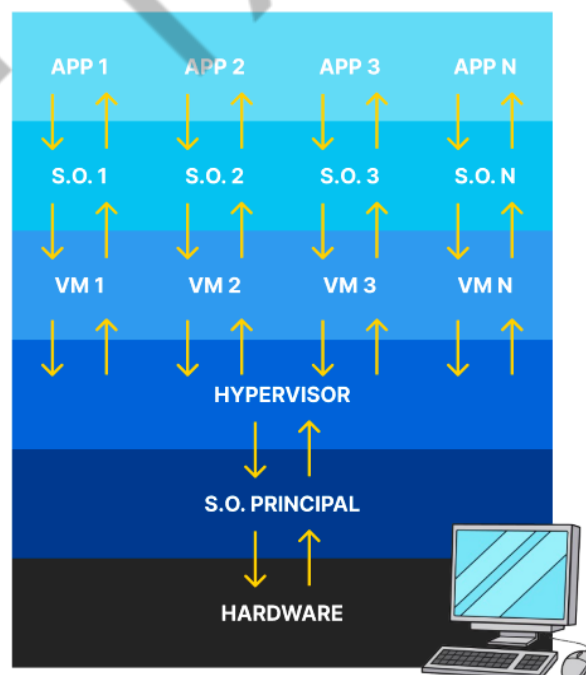


Figura 1 - Demonstração de uma arquitetura de aplicações utilizando máquinas virtuais
Fonte: Elaborado pelo autor (2024)

Além disso, teremos como base de todas essas aplicações um único sistema operacional que servirá de intermediário para o hypervisor disponibilizar recursos do sistema para os N sistemas operacionais disponibilizarem recursos para as N aplicações que estão sendo executadas dentro daquela máquina virtual. Isso consequentemente gera competição de recursos do sistema como um todo, de cada sistema isolado e de cada aplicação dentro de cada máquina virtual. Sem falar no alto custo e tempo de configuração e manutenção.

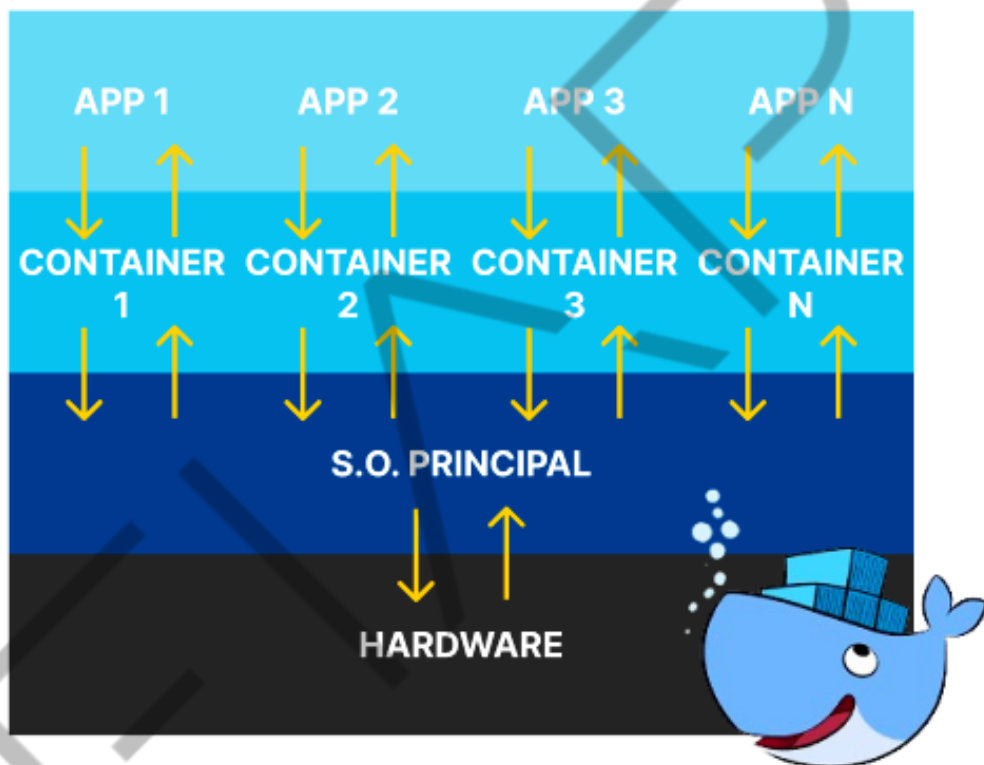


Figura 2 - Demonstração de uma arquitetura de aplicações utilizando containers
Fonte: Elaborado pelo autor (2024)

Para a utilização de containers, precisamos somente do hardware e um único sistema operacional. O hypervisor deixa de existir e entra em cena o Docker Engine (ou Docker daemon), que fará a comunicação direta e será responsável pelo gerenciamento de containers, imagens, networks e volumes do Docker.

O Docker Engine é um processo que é executado em kernel compartilhado, o qual também tem a responsabilidade de realizar a divisão de funcionalidades e recursos do sistema. Utilizando containers, temos redução de custo, tempo de manutenção e infraestrutura. Além disso, evitamos múltiplas máquinas virtuais.

Recomendo fortemente assistir às aulas de Containers para aprender como o Docker funciona.

Docker Daemon

O que é o Docker Daemon?

O Docker Daemon ou Docker Engine, é o **processo central que gerencia os containers no Docker**. Ele é responsável por tarefas como, por exemplo, criar, executar e monitorar containers, além de gerenciar imagens, redes e volumes do Docker. O Docker Engine opera em segundo plano no sistema operacional do host e interage diretamente com o Docker Client e o Registry através de uma REST API, conhecida como Docker Engine API e sendo um módulo do Docker Engine.

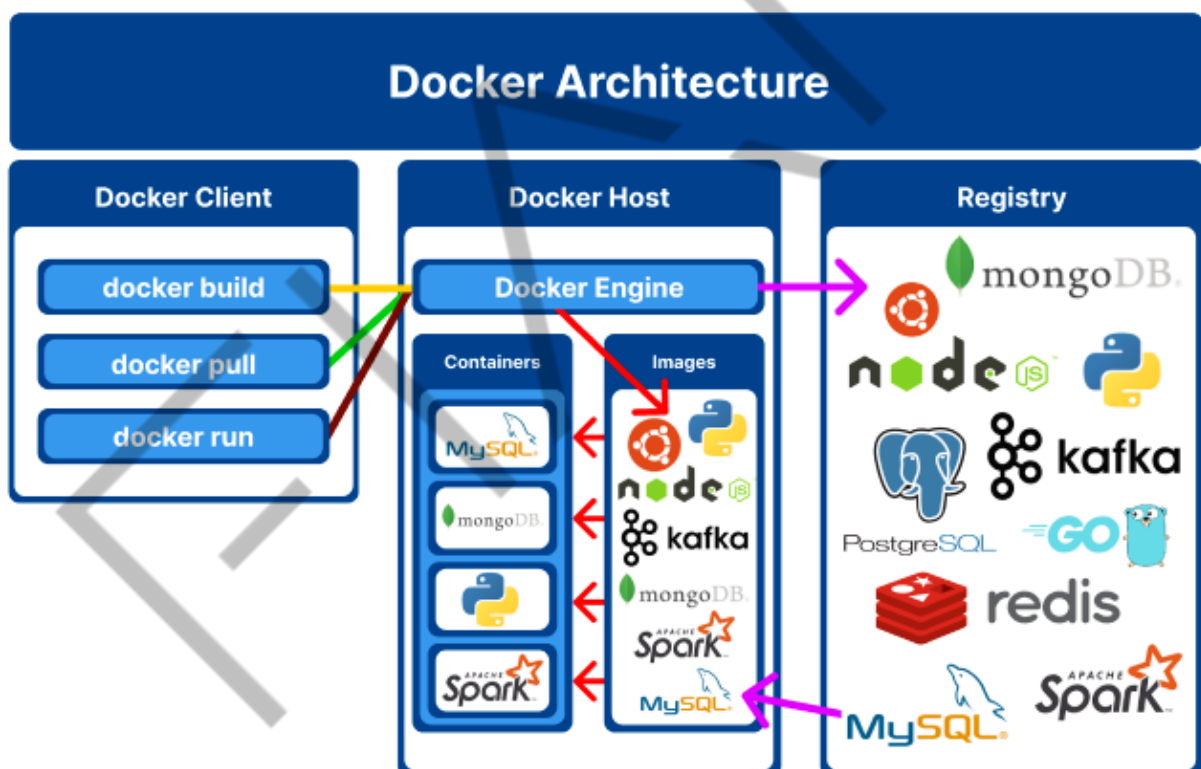


Figura 3 - Demonstração dos componentes e arquitetura do Docker para explicação do que é Docker Daemon

Fonte: Elaborado pelo Autor (2024)

Como o Docker Daemon (ou Docker Engine) funciona?

O Docker usa uma arquitetura cliente-servidor em que o Docker Client (docker) envia comandos para o Docker Daemon (dockerd). O Docker Client é a CLI

(Command Line Interface) na qual executamos comandos, como por exemplo `docker ps` ou `docker run hello-world`.

Quando executamos um comando, como por exemplo `docker run hello-world`, o Docker Client envia informações para o Docker Daemon via REST API para realizar as ações necessárias, como iniciar um container ou baixar uma imagem nova do Registry.

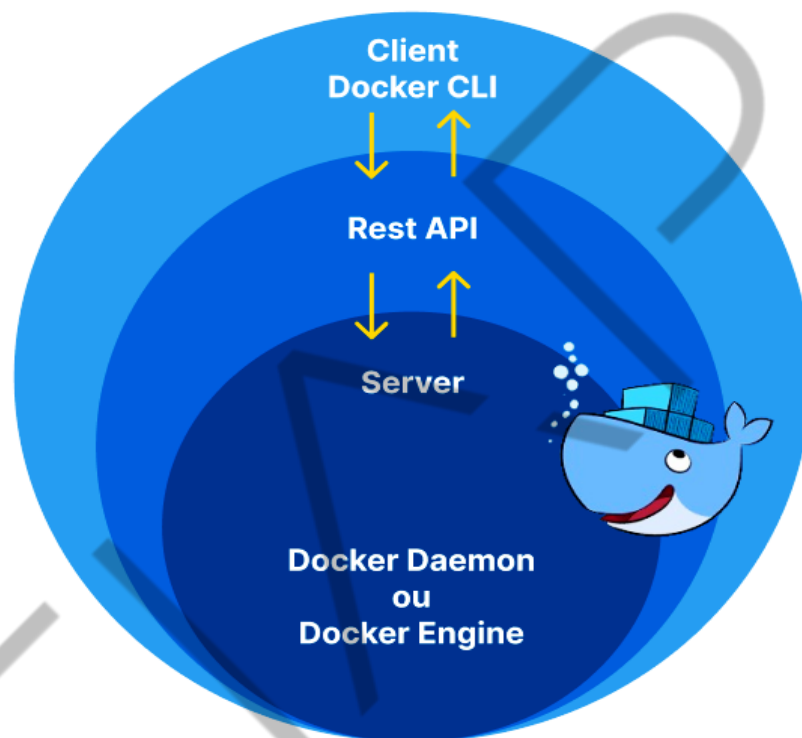


Figura 4 - Demonstração das camadas principais do Docker
Fonte: Elaborado pelo Autor (2024)

O Docker Daemon garante que os containers operem de forma isolada no sistema e, além disso, gerencia recursos locais, armazenamento e distribuição de imagens Docker, incluindo criação, execução e destruição de containers.

Namespaces e Control Groups (cgroups)

Entrando um pouco mais a fundo no Docker Engine, quando executamos um container Docker, “por trás das cenas” são criados namespaces e grupos de controle (cgroups) para o container.

São tecnologias do kernel Linux. Os namespaces têm como função isolar recursos do sistema para que cada container tenha sua própria visão do sistema. Já

os cgroups têm como função gerenciar e limitar a quantidade de recursos que os containers podem usar, permitindo que nenhum container possa consumir todos os recursos do sistema.

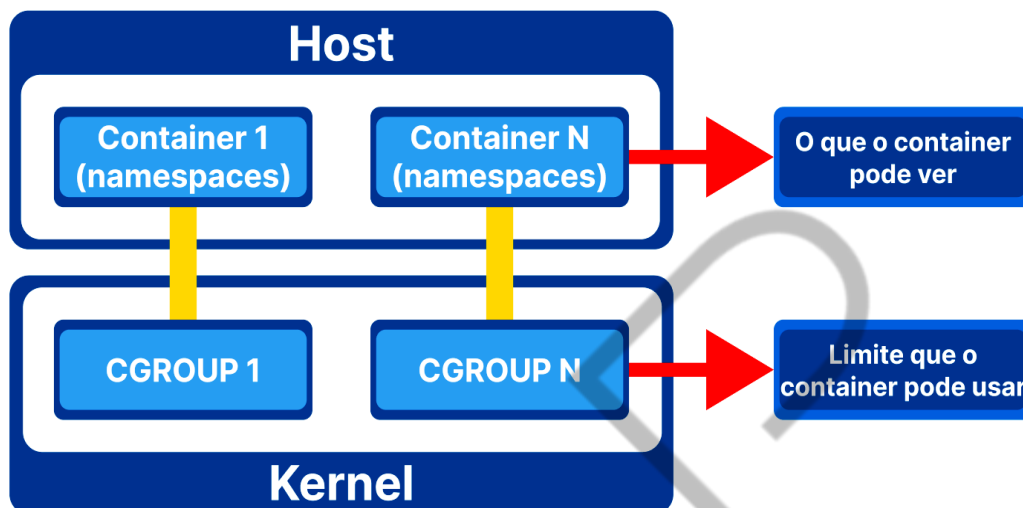


Figura 5 - Demonstração de Namespaces e Cgroups

Fonte: Elaborado pelo Autor (2024)

Sendo assim, por conta dessas ferramentas, é possível selecionarmos o quanto de recursos queremos utilizar em nossos containers, escolhendo quantidade mínima e máxima de memória e processamento, por exemplo:

```
$ sudo docker run --memory="1g" --cpus="1.0" ubuntu
```

Código-fonte 1 – Exemplo de comando utilizando gerenciamento de recursos

Fonte: Elaborado pelo autor (2024)

No código-fonte 1, estamos executando um container ubuntu, selecionando e reservando um gigabyte de memória RAM e um núcleo de processamento.

Comandos do docker

A seguir, evidenciaremos os comandos relacionados aos fundamentos do Docker, bem como outros que não foram abordados anteriormente e que podem auxiliar no dia a dia.

Visualizando containers

Para exibir todos os containers ativos:

```
$ docker ps
```

Código-fonte 2 – Exemplo de comando que exibe todos os containers ativos
Fonte: Elaborado pelo autor (2024)

Para exibir todos os containers inativos:

```
$ docker ps -a
```

Código-fonte 3 – Exemplo de comando que exibe todos os containers inativos
Fonte: Elaborado pelo autor (2024)

Executando containers

Para executar um novo container:

```
$ docker run --name <container_name> <imagem>
```

Código-fonte 4 – Exemplo de comando que executa um novo container
Fonte: Elaborado pelo autor (2024)

Para executar um container já existente:

```
$ docker start <container_id ou container_name>
```

Código-fonte 5 – Exemplo de comando que executa um container já existente
Fonte: Elaborado pelo autor (2024)

Para executar um novo container no modo iterativo:

```
$ docker run --name <container_name> -it <imagem>
```

Código-fonte 6 – Exemplo de comando que executa um novo container no modo iterativo
Fonte: Elaborado pelo autor (2024)

Para executar um container existente no modo iterativo:

```
$ docker start <container_id ou container_name>  
$ docker exec -it <container_id ou container_name> /bin/bash
```

Código-fonte 7 – Exemplo de comando que executa um container existente no modo iterativo
Fonte: Elaborado pelo autor (2024)

Para executar um novo container em segundo plano:

```
$ docker run --name <container_name> -d <imagem>
```

Código-fonte 8 – Exemplo de comando que executa um novo container em segundo plano
Fonte: Elaborado pelo autor (2024)

Para executar um container existente em segundo plano:

```
$ docker start <container_id ou container_name>
```

Código-fonte 9 – Exemplo de comando que executa um container existente em segundo plano
Fonte: Elaborado pelo autor (2024)

Parando containers

Para encerrar um container:

```
$ docker stop <container_id ou container_name>
```

Código-fonte 10 – Exemplo de comando que encerra um container
Fonte: Elaborado pelo autor (2024)

Para encerrar um container com temporizador:

```
$ docker stop -t <tempo_em_seg> <container_id ou container_name>
```

Código-fonte 11 – Exemplo de comando que encerra um container com temporizador
Fonte: Elaborado pelo autor (2024)

Para encerrar todos os containers:

```
$ docker stop -t <tempo_em_seg> $(docker ps -q)
```

Código-fonte 12 – Exemplo de comando que encerra todos os containers
Fonte: Elaborado pelo autor (2024)

Reinicializando containers

Para reinicializar um container:

```
$ docker restart <container_id ou container_name>
```

Código-fonte 13 – Exemplo de comando que reinicializa um container
Fonte: Elaborado pelo autor (2024)

Excluindo containers

Para excluir um container:

```
$ docker rm <container_id ou container_name>
```

Código-fonte 14 – Exemplo de comando que exclui um container
Fonte: Elaborado pelo autor (2024)

Para excluir todos os containers que não estão sendo usados:

```
$ docker container prune
```

Código-fonte 15 – Exemplo de comando que exclui todos os containers que não estão sendo usados
Fonte: Elaborado pelo autor (2024)

Vincular portas nos containers

Para vincular a porta de um container com a sua máquina local manualmente:

```
$ docker run -d -p <maquina>:<container> <imagem>
```

Código-fonte 16 – Exemplo de comando que executa um novo container e vincula portas
Fonte: Elaborado pelo autor (2024)

Para vincular a porta de um container com a sua máquina local automaticamente:

```
$ docker run -d -P <imagem>
```

Código-fonte 17 – Exemplo de comando que executa um novo container e vincula portas automaticamente
Fonte: Elaborado pelo autor (2024)

Para saber quais são as portas que o container está se comunicando com a sua máquina local:

```
$ docker port <container_id ou container_name>
```

Código-fonte 18 – Exemplo de comando que informa quais portas o container está utilizando
Fonte: Elaborado pelo autor (2024)

Volumes

Para vincular o diretório do container com um diretório do seu sistema operacional com o parâmetro -v (“\” é uma quebra de linha para o comando):

```
$ docker run -p <maquina>:<container> \  
-v <diretorio_maquina>:<diretorio_container> <imagem>
```

Código-fonte 19 – Exemplo de comando que executa um container, vincula portas e vincula volume com o parâmetro -v do docker
Fonte: Elaborado pelo autor (2024)

Para vincular o diretório do container com um diretório do seu sistema operacional com o parâmetro --mount (“\” é uma quebra de linha para o comando):

```
$ docker run -d --mount \
type=bind,source="$ (pwd) ",target=<diretorio_maquina>:<diretori
o_container> <imagem>
```

Código-fonte 20 – Exemplo de comando que executa um container, vincula portas e vincula volume com o parâmetro --mount do docker
Fonte: Elaborado pelo autor (2024)

Para criar um volume gerenciado pelo Docker:

```
$ docker volume create
```

Código-fonte 21 – Exemplo de comando que executa a criação de um volume gerenciado pelo docker
Fonte: Elaborado pelo autor (2024)

Para listar todos os volumes do Docker:

```
$ docker volume ls
```

Código-fonte 22 – Exemplo de comando que lista todos os volumes existentes
Fonte: Elaborado pelo autor (2024)

Para ver as informações de um volume gerenciado pelo Docker:

```
$ docker volume inspect <volume_name>
```

Código-fonte 23 – Exemplo de comando que retorna informações sobre um determinado volume criado
Fonte: Elaborado pelo autor (2024)

MERCADO, CASES E TENDÊNCIAS

Containers têm sido amplamente adotados por empresas de todos os tamanhos, desde startups até grandes corporações. Eles permitem uma implantação mais rápida e confiável de aplicações, melhoram a escalabilidade e reduzem custos de infraestrutura. Empresas como Google, Amazon, e Netflix utilizam containers para gerenciar suas infraestruturas de TI de maneira eficiente e escalável.



Figura 6 - Algumas empresas que utilizam o Docker em seus negócios
Fonte: Docker (2024)

A Netflix tem seu próprio gerenciador de containers chamado Titus, que é integrado à AWS e utiliza o Docker como runtime de containers. Esse gerenciador de containers concorre com Docker Swarm e Kubernetes (ambos criados pela dotCloud).

O Kubernetes é uma plataforma de orquestração de containers em que podemos realizar deploy de aplicações, gerenciar recursos e outras coisas. Já o Docker Swarm é uma versão mais simplificada do Kubernetes para pequenas e médias implementações.

Fica a pergunta: por que a Netflix criou seu próprio gerenciador de containers e não utilizou o Kubernetes ou o Docker Swarm? Qual foi sua motivação?

A motivação de criar o Titus foi a necessidade de suportar uma vasta escala de diversidade de cargas de trabalho, incluindo streaming, recomendações e aprendizado de máquina. Inicialmente o Titus foi construído para processamento em

lote e pode lidar com até três milhões de containers semanalmente; além disso, se destaca por sua integração com a AWS, agendamento avançado e escalabilidade robusta.

No cenário brasileiro temos como exemplo a Globo, que utiliza Kubernetes do GCP, sendo ele o GKE, como gerenciador de containers e uma plataforma PaaS (platform as a service) chamada Tsuru, que tem como objetivo facilitar deploys e dar suporte a múltiplos ambientes de execução para diversas aplicações e linguagens.

Gerenciando mais de 400 aplicações, o Tsuru suporta predominantemente linguagens como Python, JavaScript e Java. O Docker na Globo é essencial para o desenvolvimento local de novas implementações porque garante um ambiente consistente, isolado e portátil; sendo assim, configurações do host não afetam negativamente a produtividade do(a) desenvolvedor(a).

Além disso, uma das funcionalidades do Tsuru, é realizar build e deploy da aplicação por meio de um arquivo (que veremos nas aulas seguintes) chamado Dockerfile.

O QUE VOCÊ VIU NESTA AULA?

Nessa aula você aprendeu o conceito de containers e sua importância, além de diferenciar containers de máquinas virtuais e suas vantagens. Além disso, aprendeu como instalar o Docker no seu sistema operacional e executou seus primeiros comandos, incluindo o conceito e comandos utilizando volumes. Agora você tem uma base sólida de conhecimento sobre o Docker, parabéns!

Lembre-se que estamos disponíveis na nossa comunidade do Discord para tirar dúvidas e te ajudar. Além disso, o conteúdo estará sempre disponível quando você sentir a necessidade de revisitá-lo!

REFERÊNCIAS

DOCKER DOCS. **What is Docker?** 2024. Disponível em: <<https://docs.docker.com/guides/docker-overview/>>. Acesso em: 27 ago. 2024.

NETFLIX TECHBLOG. **Titus, the Netflix container management platform, is now open source.** 2018. Disponível em: <<https://netflixtechblog.com/titus-the-netflix-container-management-platform-is-now-open-source-f868c9fb5436>>. Acesso em: 27 ago. 2024.

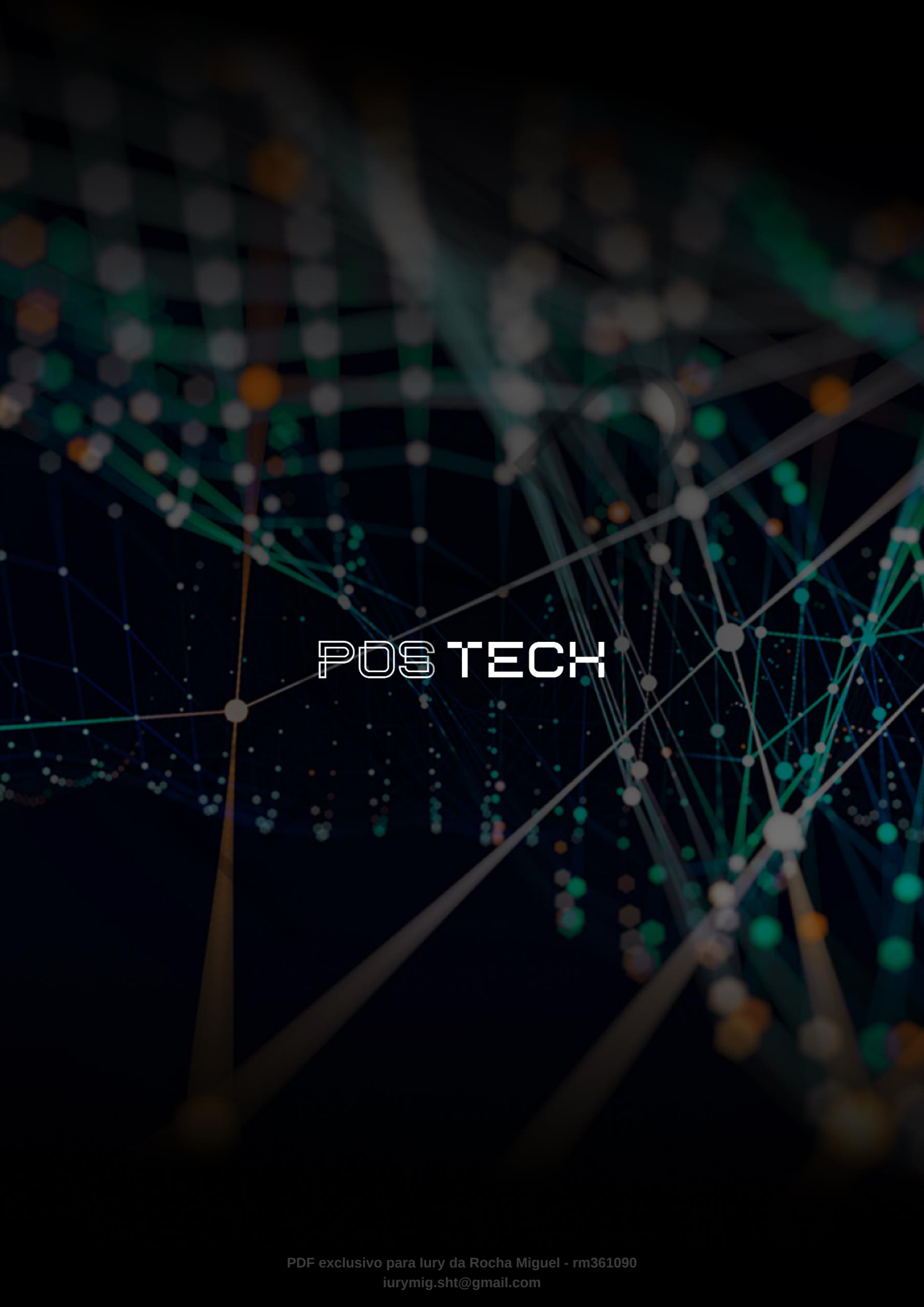
PAULO, W. **Docker e Docker Compose.** 2020. Disponível em: <<https://medium.com/@willianpaulo99/docker-e-docker-compose-bf967ffa84b3>>. Acesso em: 27 ago. 2024.

SUNDARAY, H. **Docker Architecture Explained:** Docker Client, Docker Daemon & Docker Registry. 2023. Disponível em: <<https://kodekloud.com/blog/docker-architecture/>>. Acesso em: 27 ago. 2024.

PALAVRAS-CHAVE

Palavras-chave: Containers. Docker. Virtualização. Desenvolvimento de Software. Implantação de Aplicações.

EMSE



POSTECH